

MANUAL TECNICO – PROYECTO #2

Miguel Andrés Álvarez López 202406012 – Lenguajes Formales y de programación I

REQUIERIMIENTOS PARA LA ELABORACION DE LA PROYECTO.

Esta práctica conlleva a aplicar todo lo aprendido y practicado en las primeras semanas de semestre, utilización de pseudocódigo y comprensión de lectura de archivos y estructuras. Con el fin de aplicarlos a lenguaje de programación formal con el fin de fomentar y fortalecer prácticas en lenguaje real, la practica se llevo a cabo en la aplicación para escritorios “Codeblocks” Utilizando el lenguaje C++.

QT Creator 6 (ejecución)

Se utilize la IDE QT Creator 6 (ENTERPRISE / COMMUNITY) para la realización gracias a su soporte de Interfaces graficas.



Github / Github Desktop

Se utilizo GitHub para crear, almacenar y cambiar el proyecto en tiempo real, Fue de mucha utilidad gracias a su almacenamiento en la nube y poder acceder al repositorio de manera local y remota (Para prueba en otros dispositivos). El uso de GitHub Desktop fue vital para el proyecto, ya que su fácil interfaz, la manera de realizar commits, pushes y pulls hace el proceso más fácil que utilizar la consola de Git Bash o CMD, se pueden visualizar los cambios realizados en tiempo real y la capacidad de tener varios repositorios abiertos lo hace versátil.



GitHub
Desktop

DIAGRAMA DE CLASES

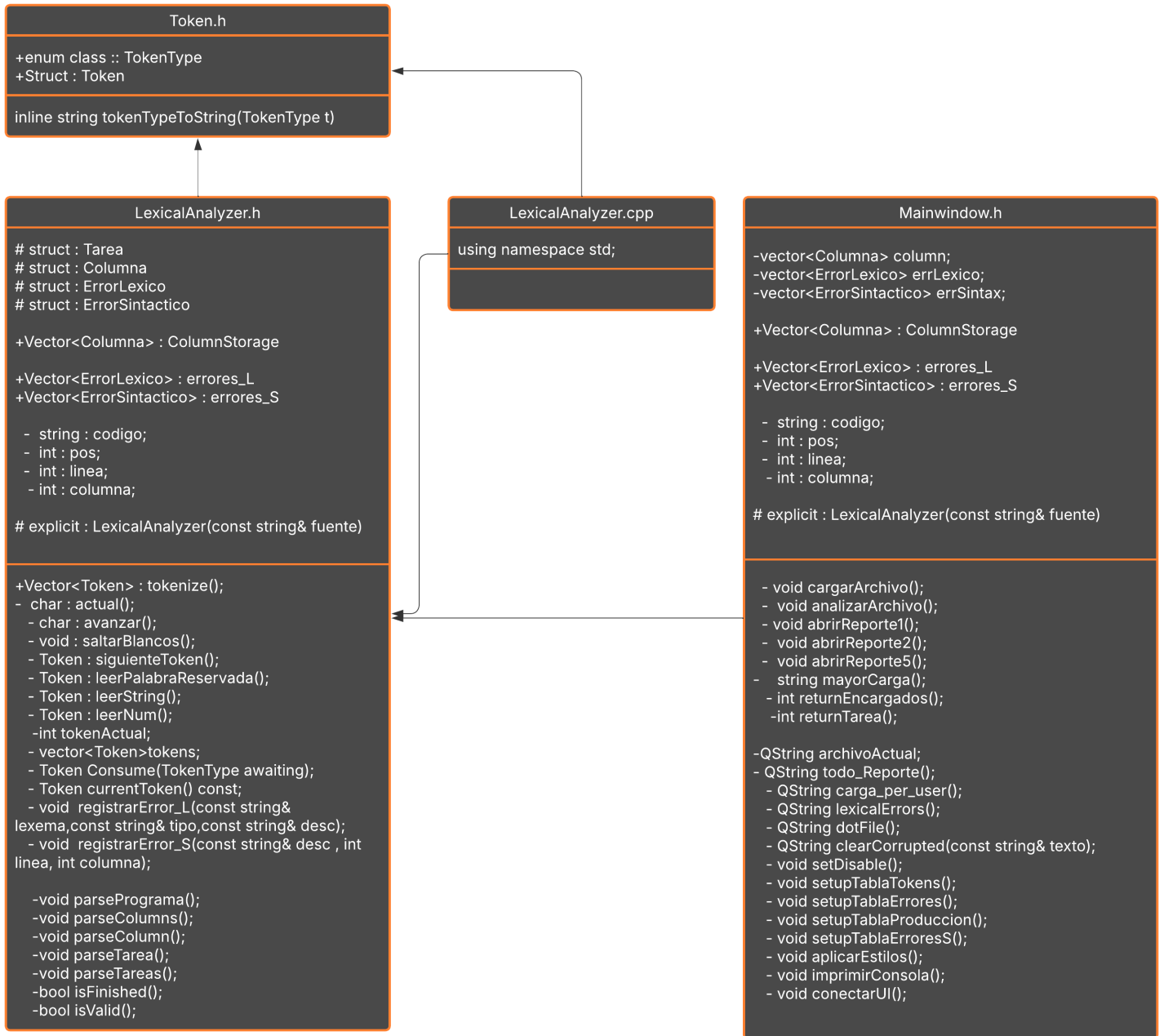
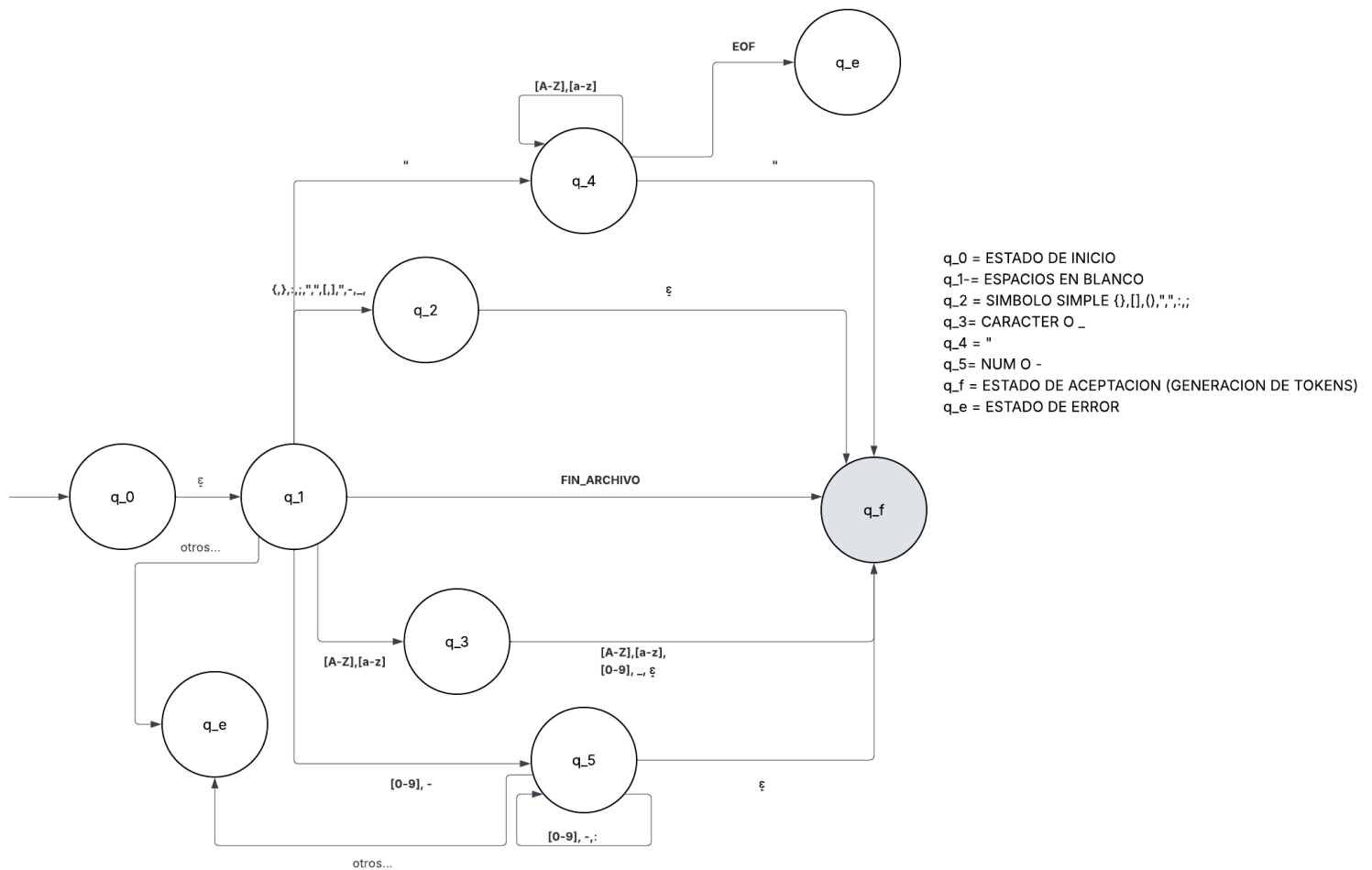


DIAGRAMA DE AFD

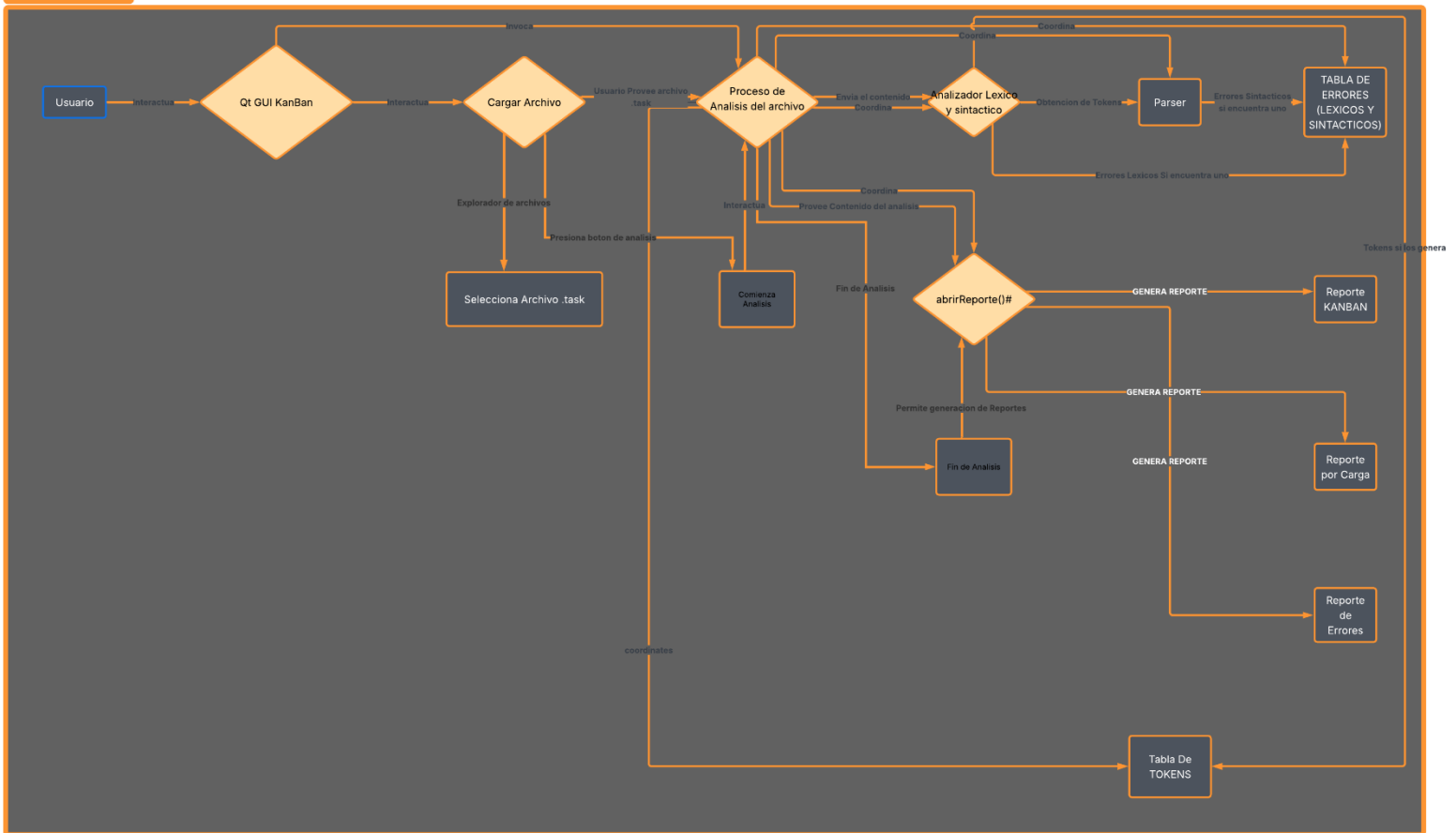


GRAMATICA LIBRE DE CONTEXTO

```
<programa> ::= TABLERO CADENA "{" <columnas> "}" ";";  
<columnas> ::= <columna> <columnas_resto>;  
<columnas_resto> ::= <columna> <columnas_resto> | ε;  
<columna> ::= COLUMNA CADENA "{" <tareas> "}" ";";  
<tareas> ::= <tarea> <tareas_resto>;  
<tareas_resto> ::= ";" <tarea> <tareas_resto> | ε;  
<tarea> ::= tarea ":" CADENA "[" <atributos> "]" ;  
<atributos> ::= <atributo> <atributos_resto>;  
<atributos_resto> ::= ";" <atributo> <atributos_resto> | ε;  
<atributo> ::= prioridad ":" <nivel_prioridad>  
                | responsable ":" CADENA  
                | fecha_limite ":" <fecha>  
                ;  
<nivel_prioridad> ::= ALTA | MEDIA | BAJA ;  
<fecha> ::= CADENA | FECHA ; // FECHA = formato YYYY-MM-DD
```

DIAGRAMA COMPONENTES UML

QTCreator KANBAN TASKSCRIPT



METODOS, VARIABLES Y PROCESOS IMPORTANTES PARA LA ELABORACION.

Token y TokenType:

TokenType Funciona para identificar tokens que se utilizaran para la lectura de un lexema e identificar el patrón al que pertenecen. Si no están incluidos en el TokenType se considerarán desconocidos para el programa. Por ello es importante declarar un token en el TokenType además de identificar la entrada servirá para leer el documento e identificar que tipo de token pertenece el Lexema

```
enum class TokenType { //TODOS LOS TIPOS DE TOKENS
    // Palabra reservada raíz
    TABLERO,

    // Delimitadores
    LLAVE_ABRE,    // {
    LLAVE_CIERRA,  // }
    PUNTO_COMA,    // ;
    //PALABRAS RESERVADAS V2
    DESCONOCIDO, //
    FIN_ARCHIVO, //
    STRING, //
    CHAR, //
    NUMERO, //
    //SIMBOLOS ESPECIALES
    COMA, //
    DOS_PUNTOS, //
    CORCH_ABRE, //
    CORCH_CIERRA, //
    //NUEVOS TOKENS
    COLUMNA,
    tarea,
    prioridad,

    ALTA,
    MEDIA,
    BAJA,
    responsable,
    fecha_limite,
    prioridades,
    Fecha,
    delimitador,
};
```

(Token.h; fila 12 a 60)

Estructuras de Datos:

Las estructuras de datos sirven para almacenar los datos leídos en sus respectivas estructuras, por ejemplo los datos que contienen un medico tendrán una estructura a seguir,

Esto se hace para que al tener que comparar valores, identificar o leer en si lo recopilado no sea un dolo de cabeza tener todo en un solo vector o estructura.

Ademas debe ser declarado 2 veces para LexicalAnalyzer.cpp y mainwindow.cpp

(LexicalAnalyzer.h; fila 8 a 48) y

(mainWindow.h; fila 45 a 51)

```
//ESTRUCTURA SINTACTICA

struct Tarea{
    string nombre_tarea;
    string nivel_prioridad;
    string fecha_limit;
    string responsable;
};

struct Columna{
    string Columna;
    //vector que contenga las tareas
    vector<Tarea> tarea_columnn;
};

struct ErrorLexico {
    string lexema;
    string tipoError;
    string descripcion;
    int linea;
    int columna;
};

struct ErrorSintactico {
    string descripcion;
    int linea;
    int columna;
};
```

```
vector<Columna> columnStorage; //almacenar tareas
// Retorna todos los tokens del archivo
vector<Token> tokenize(); //declara vector para tokenizacion y mostrar en tablas
// Errores acumulados durante el análisis
vector<ErrorLexico> errores_L; // almacena errores lexicos
vector<ErrorSintactico> errores_S; //almacena los errores sintacticos
```

```
//vectores que utilizara
vector<Columna> column;
vector<ErrorLexico> errLexico;
vector<ErrorSintactico> errSintax;
```

LexicalAnalyzer.h:

Clase que contiene los vectores para almacenar datos (Medicos,Pacientes,Citas,Diagnosticos y errores Lexicos), además de implementar todas las funciones necesarias para ser implementadas en mainwindow.cpp, sirve para ser utilizado en otras clases, inicializar y definir sus métodos para la utilización de los mismo, también utilizar los valores guardados o guardar valores en los vectores.

```
//ANALIZADOR LEXICO
class LexicalAnalyzer {
public:

    explicit LexicalAnalyzer(const string& fuente); //Constructor OBLIGATORIO
    //Constante para que no cambie
    //Contenedores ALMACENARAN LOS DATOS AL LEER EL DOCUMENTO
    vector<Medic> medStorage; //
    vector<Citas> citStorage; //
    vector<Patient> patStorage;
    vector<Diagnostic> diagStorage;

    // Retorna todos los tokens del archivo
    vector<Token> tokenize(); //declara vector para tokenizacion

    // Errores acumulados durante el análisis

    vector<ErrorLexico> errores;

private:
    //METODOS Y VARIABLES ABSTRACTAS
    string codigo; // Contenido completo del archivo
    int pos; // Posición actual en el string
    int linea; //EN QUE LINEA
    int columna; //EN QUE COLUMNA
    char actual(); // Carácter en pos actual
    char avanzar(); // Avanza y retorna el siguiente carácter
    void saltarBlancos(); // Ignora espacios, tabs, saltos de línea
    Token siguienteToken(); // AFD principal
    Token leerPalabraReservada(); // Reconoce HOSPITAL (y futuras), retorna
    Token leerString(); //Lectura de strings
    Token leerNum(); //lectura de numeros o guiones
    void registrarError(const string& lexema,const string& tipo,const string& desc);

    //REGISTRAR DATOS EN LEXICALANALYZER
    void registrarMedicos(const vector<Token>& tokens, int& pos);
    void registrarCitas(const vector<Token>& tokens, int& pos);
    void registrarPacientes(const vector<Token>& tokens, int& pos);
    void registrarDiagnosticos(const vector<Token>& tokens, int& pos);

};
```

LexicalAnalyzer-helpers

Aca es una colección de múltiples funciones para lexicalAnalyzer.cpp

```
// --- Helpers ---

char LexicalAnalyzer::actual() {
    if (pos < (int)codigo.size()) return codigo[pos];
    //si la posicion es inferior al codigo retorna la posicion actual
    return '\0'; //de lo contrario si no lee nada retorna cero //devuelve un cahr
}

char LexicalAnalyzer::avanzar() {
    //FUNCION PARA MOVER A LA SIGUENTE FILA
    char c = actual(); //trae la posicion actual
    pos++; //incrementa la posicion actual
    if (c == '\n') { linea++; columna = 1; } //si es un salto de linea
    else { columna++; }
    return c; //retorna actual
}

void LexicalAnalyzer::saltarBlancos() { //salta espacios en blanco
    while (pos < (int)codigo.size() && isspace((unsigned char)actual())) //mientras la posicion sea inferior al codigo Y ENCUENTRE UN ESPACIO EN BLANCO
        avanzar();
}

//al detectar un error lexico
void LexicalAnalyzer::registrarError(const string& lex, const string& tipo, const string& desc) {
    //llama al vector de errores y mete el error lexico
    errores.push_back({ lex, tipo, desc, linea, columna });
}
```

1. Actual(): esta función retorna un char como respuesta si la posición es inferior al tamaño del lexema y saber DONDE esta leyendo actualmente en el documento .med
2. Avanzar(); esta function se complementa con Actual();, retornando el valor posterior incrementando la posición donde esta en el documento.
3. saltarBlancos(); salta espacios en blancos.
4. registrarError(): Si detecta un error léxico, lo envia al vector de ErrorLexico para guardarlo en su uso posterior

LexicalAnalyzer::leerString y LeerNumeros

Funciones como su nombre lo dicen sirven principalmente para leer entradas como strings o números (IMPORTANTE PARA EL REGISTRO DE FECHAS)

```
//DECLARACION DE FUNCION PARA LECTURA DE STRINGS
Token LexicalAnalyzer::leerString(){
    int linIn = linea;
    int colIn = columna;
    string lexema; //llegara con cualquier contenido

    avanzar(); // Saltara la comilla inicial

    while (pos < (int)codigo.size() && actual() != '"') {
        lexema += avanzar();
    }

    // Verificar que se encontró la comilla de cierre
    if (actual() == '"') {
        avanzar(); // Saltar la comilla final
        return { TokenType::STRING, lexema, linIn, colIn };
    } else {
        registrarError(lexema, "String mal formado", "Falta comilla de cierre");
        return { TokenType::DESCONOCIDO, lexema, linIn, colIn };
    }
}

Token LexicalAnalyzer::leerNum(){ //LECTURA DE NUMEROS
    int linIn = linea;
    int colIn = columna;
    string lexema;

    while (pos < (int)codigo.size() && (isdigit((unsigned char)actual()) || actual() == '-' || actual() == '.')) { //si lee numeros y tienen los siguientes digitos
        lexema += avanzar();
        //Si encuentra que es un digito y contiene los simbolos especiales
    }

    if (lexema.find('.') != string::npos) {
        return { TokenType::DATE, lexema, linIn, colIn };
    } else if (lexema.find(':') != string::npos) {
        return { TokenType::HOUR, lexema, linIn, colIn };
    } else {
        return { TokenType::NUMERO, lexema, linIn, colIn };
    }
}
```

LexicalAnalyzer::siguienteToken()

Esta función es el AFD principal que se encarga de leer los tokens según el lexema que se introduzca, según su posición y la entrada devuelve una estructura de tipo TOKEN con lo que se leyó., también tiene funciones importantes como leerNum, RegistrarError

```
Token LexicalAnalyzer::siguienteToken() { //PARA SIGUIENTE TOKEN
    saltarBlancos();

    int linIni = linea; //en que linea
    int colIni = columna; //en que columna

    if (pos >= (int)codigo.size())
    |   return { TokenType::FIN_ARCHIVO, "EOF", linIni, colIni }; //Si la posicion es mayor o igual al tamaño del código actual regresa el token FIN?ARCHIVO

    char c = actual(); //RETORNO DEL ACTUAL

    // Delimitadores de un solo carácter
    if (c == '{') { avanzar(); return { TokenType::LLAVE_ABRE, "{", linIni, colIni }; } //SI CH es un Bracket { avanza y retorna que es una llave
    if (c == '}') { avanzar(); return { TokenType::LLAVE_CIERRA, "}", linIni, colIni }; } //SI CH es un Bracket { avanza y retorna que es una llave
    if (c == ';') { avanzar(); return { TokenType::PUNTO_COMA, ";", linIni, colIni }; } //SI CH es un SEMICOLON ; avanza y retorna que es una SEMICOLON
    if (c == ':') { avanzar(); return { TokenType::DOS_PUNTOS, ":", linIni, colIni }; }
    if (c == ',') { avanzar(); return { TokenType::COMA, ",", linIni, colIni }; }
    if (c == '[') { avanzar(); return { TokenType::CORCH_ABRE, "[", linIni, colIni }; }
    if (c == ']') { avanzar(); return { TokenType::CORCH_CIERRA, "]", linIni, colIni }; }
    if (c == '"') { return leerString(); } // Nueva función para strings

    //reconocimiento de numeros
    if(isdigit(c) || c=='-'){
    |   return leerNum();
    }

    // Palabra reservada o identificador
    if (std::isalpha((unsigned char)c) || c == '_') //SI ES PALABRA RESERVADA O ID
    |   return leerPalabraReservada();

    // Carácter no reconocido → error léxico
    std::string lex(1, c);
    avanzar();
    registrarError(lex, "Carácter ilegal",
    |   |   |   |   "El carácter '" + lex + "' no pertenece al alfabeto de MedLang.");
    return { TokenType::DESCONOCIDO, lex, linIni, colIni };
}
```

LexicalAnalyzer::leerPalabraReservada

Esta función lee si el lexema es igual a una palabra reservada devuelve un Token para ser almacenado con la información de que tipo es y su ubicación en el archivo

LexicalAnalyzer::tokenize

```
//CREACION DE TOKENS
vector<Token> LexicalAnalyzer::tokenize() {
    vector<Token> tokens; //invoca al vector de tokens de LexicaAnalyzer
    pos = 0; linea = 1; columna = 1;
    //Limpieza de vectores
    columnStorage.clear();
    errores_L.clear();
    errores_S.clear();

    qDebug() << "Tamaño del código:" << codigo.size() << "bytes";
    //Parseo GENERAL
    while (true) {
        Token t = siguienteToken(); //lee os tokens
        if (t.tipo == TokenType::FIN_ARCHIVO) break;
        if (t.tipo != TokenType::DESCONOCIDO)
            tokens.push_back(t);
    }
    //Parseo de Columnas, GENERA LOS TOKENS PARA LOS VECTORES
    int posTok = 0;
    qDebug() << "Iniciando registro de columnas. Total tokens:" << tokens.size();

    while (posTok < (int)tokens.size()) {
        qDebug() << " posTok:" << posTok
            | << "Token:" << QString::fromStdString(tokenTypeToString(tokens[posTok].tipo));

        if (tokens[posTok].tipo == TokenType::TABLERO) {
            qDebug() << "Encontrado TABLERO en posición" << posTok;
            posTok++; // Saltar TABLERO

            // Saltar el nombre del tablero (STRING)
            if (posTok < (int)tokens.size() && tokens[posTok].tipo == TokenType::STRING) {
                qDebug() << " Saltando nombre:" << QString::fromStdString(tokens[posTok].lexema);
                posTok++;
            }

            // Ahora posTok apunta a LLAVE_ABRE
            registrarColumnas(tokens, posTok);
            qDebug() << "Volvió de registrarColumnas, posTok=" << posTok;
        } else {
            posTok++;
        }
    }
}
```

Esta función genera los tokens. Limpia los vectores de almacenamiento y genera los tokens según su tokenType

LexicalAnalyzer::registrar...

Funcion que al leer el archivo registra el token que se ingreso y se debe agregar a un vector del mismo tipo, existe Columna únicamente para este proyecto. Guarda los datos leídos por medio del lexema y los envia al vector correspondiente.

mainwindow.h y mainwindow.cpp:

Clase que contiene los componentes, métodos y funciones para la pestaña de interfaz grafica del proyecto, lo cual es muy crucial para la ejecución visual del proyecto. Este estará relacionado con LexicalAnalyzer para el almacenamiento de datos y su posterior uso en la parte grafica.

Principalmente las funciones privadas son las que realizaran las tareas, funciones o serán las variables de declaración importantes.

Para el archivo .cpp es donde se ejecutaran las funciones necesarias o se declararan correctamente para su uso posterior, en este archivo es donde se utiliza el `setUpUI()`; para hacer funcionar la interfaz grafica.

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QTableWidget>
#include <QTextEdit>
#include <QPushButton>
#include <QLabel>
#include <QTabWidget>
#include <QStatusBar>
#include <vector>
#include <string>
#include "LexicalAnalyzer.h"

//TODO LO QUE MANEJARA LA INTERFAZ
QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow() override;

private slots:
    //FUNCIONES POR EVENTOS POR MEDIO DEL GUI
    void cargarArchivo();
    void analizarArchivo();
    void abrirReporte1(); //TAREAS
    void abrirReporte2(); //CARGA
    void abrirReporte3(); //ERRORES

    //UNUSED

private: //DONDE SE ALMACENARAN TODAS FUNCIONES QUE NO RESPONDEN A EVENTOS POR MEDIO DEL GUI
    //ESTAS FUNCIONES NO RESPONDEN A LOS BOTONES DEL UI, SOLAMENTE FUNCIONAN DE MANERA INTERNA
    Ui::MainWindow *ui; //Puntero a la interfaz generada (NO MOVER)

    //FUNCIONES DE RETORNO
    //vectores que utilizara
    vector<Columna> column;
    vector<ErrorLexico> errLexico;
    vector<ErrorSintactico> errSintax;

    //Funciones a utilizar
    string mayorCarga(); //mayor carga por usuario
    int returnEncargados(); // retorno de numero de encargados
    int returnTareas(); //retorno de tareas
    //
    QString archivoActual;
    //NEW REPORTES
    QString todo_Reporte(); //REPORTES DE TAREAS
    QString carga_per_user(); //REPORTES DE CARGA POR USUARIO
    QString lexicalErrors(); // REPORTES DE ERRORES
    QString dotFile(); //ARBOL SINTAXIS .DOT
    QString clearCorrupted(const string& texto); //limpiar comillas pedorras

    void setDisable();
    void setupTableTokens();
    void setupTableErrores();
    void setupTableProduccion();
    void setupTableErroresS();
    void aplicarEstilos(); //ESTILOS
    void imprimirConsole();
    void conectarUI();
};
#endif // MAINWINDOW_H
```

Mainwindow (Generar reportes y estructura de reportes)

Son 2 funciones que se complementan, para la generación del archivo HTML o .DOT se utiliza la función `abrirArchivoN()`; donde será el encargado de crear un archivo y contener lo estructurado en la función para retornar un archivo HTML.

```
void MainWindow::abrirReporte3() { //REPORT DE CITAS

//GENERAR CITAS
qInfo() << "Generando reporte Citas ";

if (archivoActual.isEmpty()) {
    qWarning() << "ERROR: No hay archivo cargado";
    QMessageBox::warning(this, "Error", "Primero carga un archivo .med");
    return;
}

if(!archivoActual.isEmpty()){
    QString ruta = QFile::fromRawFd(archivoActual).absolutePath();
    QString path = ruta + "/reporte_citas.html"; //Ruta donde se almacenara

    QDir dir(ruta);
    if(!dir.exists()){
        QMessageBox::critical(this, "Error", "La ruta: " + ruta + " No existe");
        return; //defunde la funcion
    }

    QString htmlContents = citasReport(); //genera el reporte con datos
    QFile archivo(path); //guarda el archivo
    if(archivo.open(QIODevice::WriteOnly | QIODevice::Text)){
        QTextStream out(&archivo);
        out << htmlContents;
        archivo.close();
        //POPUP
        QMessageBox::information(this, "Citas!", "Reporte de Citas generado correctamente.");
    }else{
        QMessageBox::information(this, "ERROR, Reporte Citas!", "Ocurrio un error al GENERAR el archivo");
    }
}

}
```

Estructura HTML:

```
QString MainWindow::citasReport(){ //HTML CITAS
//EN LA GENERACION DE REPORTE SE REALIZARA LA INSPECCION DE SI EXISTEN DATOS QUE COINCIDEN O NO

    QString html = R"(
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Agenda de Citas</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            margin: 40px;
            background-color: #f5f5f5;
            color: #333;
        }
        .container {
            max-width: 1200px;
            margin: 0 auto;
            background-color: white;
            border-radius: 10px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.1);
            padding: 30px;
        }
        h1 {
            color: #2c3e50;
            border-bottom: 3px solid #3498db;
            padding-bottom: 10px;
        }
        .fecha {
            color: #7f8c8d;
            text-align: right;
            margin-bottom: 20px;
        }
        table {
            width: 100%;
            border-collapse: collapse;
            margin-top: 20px;
        }
        th {
            background-color: #239923;
            color: white;
            padding: 12px;
            text-align: left;
        }
    )";
```

Tokens <VECTOR>

Como se utiliza el proyecto anterior como molde, se necesita aguardar los tokens registrados por lo que existe el vector dfe tokens el cual como su nombre lo indica guardara todos los tokens generados y se utilizaran para el Parseo Sintactico.

```
vector<Token>tokens; //guardar tokens para syntax
```

Consume y parse...

Parse:

El parseo es para definir la estructura de cada parte del Programa, dependiendo de como sea la producción de Tareas, columnas o columna y el programa en general. Se lleva de la mano con TOKENIZE

```
void LexicalAnalyzer::parsePrograma(){
    Consume(TokenType::TABLERO);

    Token name = Consume(TokenType::STRING); //CHAR
    Consume(TokenType::LLAVE_ABRE);
    parseColumns();
    Consume(TokenType::LLAVE_CIERRA);
    Consume(TokenType::PUNTO_COMA);

    qDebug() << "Parseo de Tablero Completado";
}
```

```
//SINTAX PARSEO

this->tokens = tokens;
tokenActual =0;
try {
    parsePrograma();//Inicio de parseo
    if(tokenActual < (int)tokens.size()){
        registrarError_S("ERROR,TOKENS SIN PROCESAR",linea,columna);
    }
} catch (exception e) {
    qWarning() << "Error en la lectura Sintactica" << e.what();
}

//!
qDebug() << "=== TOKENIZE END ===";
qDebug() << "Columnas registradas:" << columnStorage.size();

return tokens;//retorno
///! DEBUGG
```

Consume:

Consume es el mas importante ya que depende del parseo y como este estructurado el mismo, detecta si el token actual esta dentro del rango de tokens analizados, y en caso de que se obtenga el token esperado aumenta la posición del token actual y retorna el token, de lo contrario responderá con un error Sintactico.

```
Token LexicalAnalyzer::Consume(TokenType await){
    //SI EL TOKEN ACTUAL ESTA UBICADO FUERA DE RANGO
    if(tokenActual >= (int)tokens.size()){
        registrarError_S("ERROR,TOKENS SIN PROCESAR",linea,columna);
        //retorna a la caja de tokens
        return Token{TokenType::DESCONOCIDO,"?",linea,columna};
    }

    Token t = tokens[tokenActual]; //Token actual lo tomara
    if(t.tipo == await){ //SI EN CASO DE QUE EL TIPO DE TOKEN SEA EL ESPERADO
        tokenActual++;
        return t; //RETORNA TOKEN ACTUAL SUMADO
    }else{
        registrarError_S(tokenTypeToString(await) + ", Token Diferente al esperado",t.linea,t.columna);
        return Token{TokenType::DESCONOCIDO,"",0,0};
    }
}
```

```
void LexicalAnalyzer::parseColumns(){
    parseColumn(); //DEBE HABER UNA COLUMNA
    while (tokenActual < (int)tokens.size() &&
           tokens[tokenActual].tipo == TokenType::COLUMNA) {
        parseColumn(); // <- avance
    }
    qDebug() << "Parseo de Columnas Completado";
}

void LexicalAnalyzer::parseColumn(){
    Consume(TokenType::COLUMNA);
    Token name = Consume(TokenType::STRING); //CHAR
    Consume(TokenType::LLAVE_ABRE);
    parseTareas();
    Consume(TokenType::LLAVE_CIERRA);
    Consume(TokenType::PUNTO_COMA);
    qDebug() << "Parseo de Columna Completado";
}
```